

RMC — Reconfigurable DDR5 Memory Controller

Architecture & Design Decisions — v1.9.8

JESD79-5 | AXI4 Host Interface | DFI 5.2 PHY

4GB single-channel single-rank anchor config → 46b address, 8ch, 8rank, 64Gb×3DS-8H = 64TB ceiling

RTL style: SystemVerilog, fully parameterized, zero hardcoded widths

Architecture status: complete | RTL status: skeleton not started

In Scope

- DDR5 command pipeline: split, translate, buffer, schedule
- CAM-searched write/read buffers + RAW hazard bypass
- 5-stage bank-aware SJF command scheduler
- Per-bank / per-rank / global JEDEC timing state
- 6 sub-FSM maintenance engine (init, refresh, ZQ, RFM, power mgmt, thermal poll)
- Address hierarchy: Ch → Subch → Rank → BG → Bank → Row → Col

Out of Scope

- DDR PHY — DFI 5.2 is the fixed contract boundary
- AXI masters / interconnect upstream of the host port
- AXI WRAP/FIXED bursts, narrow/unaligned transfers
- Exclusive access, QoS ordering
- Runtime DIMM discovery (OQ-20, closed out of scope)

Baseline Anchor Config

4GB ▪ single channel ▪ single rank
32b address ▪ scales to 46b / 8ch / 8rank / 3DS-8H

BL16-Only Burst Model

A 64B AXI-aligned INCR burst = exactly one BL16 burst on a 32b DDR5 subchannel. One AXI beat-group $\equiv$ one DRAM burst, always — no fragment bookkeeping anywhere downstream.

Deliberately Narrow AXI Feature Set

INCR only, 64-bit data, multiple outstanding IDs — but no narrow/unaligned, no exclusive access, no QoS. Every cut feature added verification surface without adding value for a DRAM controller.

Exactly One Clock-Domain Crossing

CIF = AXI clock. MC core = MC clock. The *only* meeting point is one async-FIFO pair (req + resp). 100% of CDC verification burden concentrated into one well-understood primitive.

Valid-Credit Everywhere Inside MC Core

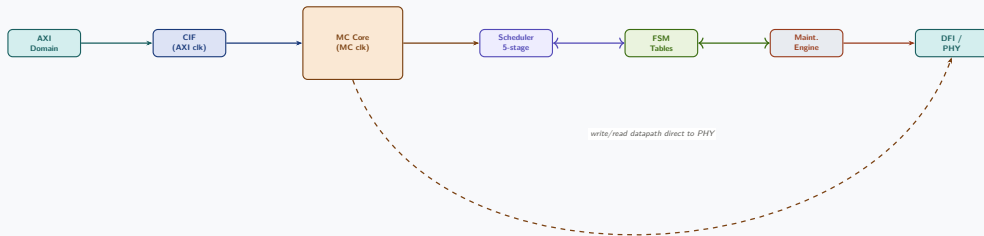
No combinational ready anywhere intra-MC — that's exactly what breaks timing closure at frequency. Every path uses registered credit counters instead. AXI ports are the one exception (spec-mandated valid-ready).

Absolute Deadlines, Not Countdown Counters

`next_legal = gc + delay` stored once; every cycle: `eligible = (gc - next_legal)[MSB]==0`. Same trick as TCP sequence-number wraparound. Collapses ~200 decrementers into ~100 static comparators against one shared free-running counter.

Registered `can_*` Flags, Off the Critical Path

Even the comparison above is computed in the background every cycle and stored as a plain bit. Scheduler Stage 2 *only reads* these flags — zero subtractors in the scheduling critical path.



Reading the flow: host requests enter left, get split/hashed/ reordered in CIF, cross the single CDC boundary into MC Core (buffers, TCAM search, RAW bypass), get arbitrated by the Scheduler against state held in the FSM Tables, with the Maintenance Engine injecting refresh/ZQ/RFM/power/thermal events and owning the DFI mux until handoff, out to DFI/PHY and the DRAM itself.



AMU replaces the plain Address Translator

Optional per-field XOR hash ($\text{hashed} = \text{raw} \oplus (\text{raw} \gg \text{shift})$) before extracting ch/rank/bg/bank/row/col. Rank + channel hashed and pushed to address MSBs (spread traffic); bg/bank/row/col left un-hashed (preserve row-hit locality — what read latency actually depends on). Split-field extraction supports arbitrary channel-interleave granularity, CSR-configurable at runtime.

WR_TCAM

Full address match: {bg,bank,row,col}

Job: exact-match RAW hazard detection — does an incoming read collide with a pending write?

24,576T baseline

RD_TCAM

Ternary match: {bg,bank} only — row/col carried, not searched.

Job: feed Scheduler Stage 1's per-bank pre-filter — only needs "is a request waiting for this bank," not the exact address.

1,920T baseline

Status Registers — single source of truth for occupancy

valid | status | age (+merge_pending for reads). TCAM match is gated by status.valid — invalid rows go electrically dark (dynamic power win at low occupancy). age triples as multi-hit tie-break (newest wins), starvation check, and SJF cost. Two Watermark Buffer Managers own their TCAM+status pair outright; Scheduler is READ ONLY.

RAW Bypass Manager & Hold-Forward

Forwarding pending write data straight into a colliding read

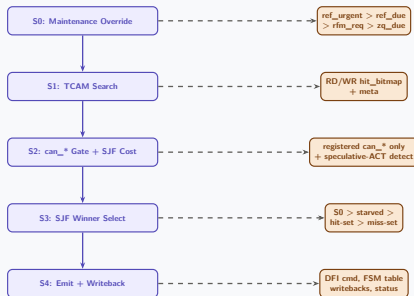


Timing (zero penalty on the common paths)

Cycle 0: WR_TCAM searched combinationaly, parallel with RD alloc — zero penalty on a miss. Cycle 1: mask-coverage check decides full hit / partial overlap / miss. **Full hit** → forward WDB data directly, no DRAM access. **Partial overlap** → issue DRAM fetch immediately (no stall), tag merge_pending, Merge Unit combines WDB+DRAM at return — same cost as a plain miss. **Hit valid only if** `wr_age <= rd_age` — a write arriving after the read doesn't count (prevents stale-data forwarding).

Hold-Forward: exactly 2 slots, proven not 3

Only two sources can produce a response in the same cycle (RAW bypass path, real DRAM return) — each rate-limited to 1/cycle by construction. A 3rd simultaneous collision is impossible, not just unlikely. 2-deep is the exact minimum *and* maximum ever required.



NOP-cycle priority (when `winner_valid==0`): 1) opportunity REFsb (correctness/spread first) 2) speculative prefetch ACT (TCAM-confirmed, zero mispredict path) 3) WR-partition drain 4) true NOP. **Starvation** is staggered ($\text{age} \geq \text{THR} + \text{entry_idx}$) so at most one starved miss can fire per cycle — no collision between two "must-service" candidates.

Per-Bank

$16 \times N_RANKS$ rows. 8-state bank FSM, open row, `next_{cas,pre,act,ref}` + `can_*` pairs. Old pending-bits removed — redundant once TCAM hit/state covers them.

Per-Rank

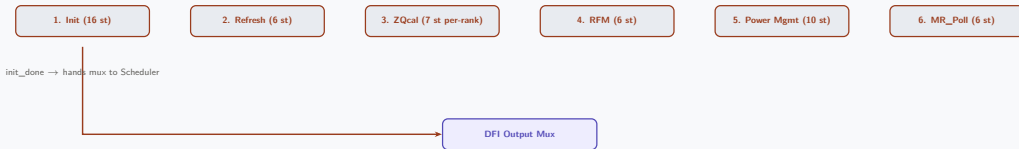
N_RANKS rows. Rank FSM, REFab/ZQcal/PDX/SRX deadlines, leaky-bucket refresh credits, per-bank RAA[16] for RFM, MR4 thermal poll state.

Global Timing

1 instance. Cross-bank `tRRD_S/tCCD_S`, `tFAW` ring buffer, per-bank-group `tRRD_L/tCCD_L/tWTR_L` family.

Bank Activity Counter — small table, heavily shared

`count` | `dirty` per bank. ME targets least-loaded bank for opportunistic refresh; Power Mgmt uses `count==0` across all banks as its power-down gate; Scheduler Stage 3 prefers draining high-count banks first.



Internal priority: $\text{ref_urgent} > \text{ref_due} > \text{rfm_req} > \text{zq_due}$ — mirrors Scheduler Stage 0 exactly. **Refresh FSM** runs a leaky-bucket credit counter, targets least-loaded bank for REFsb unless a per-bank watchdog ($\text{uncovered} > \text{tREFI} \times 32$) forces an overdue bank; fires free *opportunity refresh* on true NOP cycles. **MR_Poll** reads MR4 TUF — $\text{TUF}=1$ ($>85^\circ\text{C}$) halves effective tREFI; response arrives via a *sideband* from Read Data Path, not the response FIFO. **DFI mux:** `init_done` is a one-way latch — Init FSM drives DFI until it fires, then Scheduler drives it forever after.

RD/WR Bank Partitioning

Banks split into a read-half / write-half that rotate every WINDOW_SIZE cycles. Read and write never share a bank inside a window, so tRTW/tWTR simply don't apply within it — the turnaround penalty is paid once, at the rotation boundary, amortized across the whole window.

Speculative Prefetch ACT — NOP-cycle only, TCAM-confirmed

Fires only when `winner_valid==0` (a true NOP). If an active bank is approaching its column boundary (`COL_MAX - cur_col <= BL_BYTES`), predict {same rank, same bg, bank+1, same row} — but only emit the ACT if RD_TCAM confirms a real request already exists for that bank *and* the row matches *and* `bank_act_count > 0` *and* `can_act + can_faw` both hold. No mispredict path exists — worst case is a wasted FAW slot if the row later conflicts, nothing worse.

`bank_act_count` dual-use: 0 → no speculation, closed-page policy ▪ 1-3 → speculate if boundary imminent, adaptive page ▪ ≥ 4 → speculate aggressively, open page. New hardware cost: one comparator per active bank + one mux — zero new tables, FSMs, or buffers.

Parameter	Value	Why
N_WR_ENTRIES	$2-3 \times N_{RD}$	Writes drain in background; bigger WR buffer $\rightarrow$ lower RD latency at unchanged bandwidth
GC_WIDTH	20b baseline	Half-range 524,288 cyc vs worst-case gap $\sim 12,480$ cyc $\rightarrow 42\times$ margin (TCP-style wraparound compare)
ROB_WATERMARK	37 cyc	Measured worst-case full round trip (row miss, no REF)
REF-stall timeout	76 cyc	$2\times$ ROB watermark — "wait for in-flight CAS," not abort-and-retry
RD_STARVATION_THR	12,480 cyc ($9\times t_{REFI}$)	Read latency is user-visible; short leash
WR_STARVATION_THR	37,440 cyc ($3\times RD$)	Writes already drained preferentially — this is a backstop only
FIFO_DEPTH	16 (both async FIFOs)	$\sim 2\times$ CDC+pipeline round-trip latency
TCAM budget	$38,784T \approx 6,464$ SRAM-bit eq.	RD_TCAM (ternary, 2-field) far cheaper than WR_TCAM (exact, 4-field) — payoff of splitting by search semantics

R1 — The Scheduler never writes state directly

Only through Stage 4's committed writebacks. Stages 0–3 are pure decision logic — nothing they compute is visible anywhere else until Stage 4 commits it.

R2 — Every register has exactly one owner

Status registers → watermark managers. FSM tables → Stage 4 (+ ME for maintenance events). `timing_reg_file` → CSR. Scheduler's access to anything it doesn't own is READ ONLY, always.

R3 — State changes only via a committed command

No speculative state mutation anywhere — even speculative-ACT only fires once TCAM has confirmed the request it's speculating about actually exists.

R4 — Everything else is read-only

If a block isn't the documented owner of a piece of state, it reads it and nothing more.

Architecture Rating (v1.9.2+, unchanged through v1.9.8)

Buffers, timing model, RAW bypass, scheduler: **9/10**
vs. industry practice: **8/10**
vs. typical capstone: **significantly above**

TCAM split, registered can_ flags, credit-based CDC, SJF scheduling, bank pipelining, staggered starvation all match or exceed patterns in real DDRC IP (Synopsys DDRC, ARM DMC-620 class).*

Before RTL (priority order)

1. Patch stale LaTeX docs (BUG-02..07 — pending bits, hardcoded depths, RAW partial-hit description)
2. OQ-19b — channel-interleave granularity via Python traffic-trace script
3. RTL skeleton — top-level hierarchy, parameter propagation

Only open architectural question left: OQ-19b. Everything else in the v1.0.1–v1.9.8 history is closed.